



Facultad de
Ciencias
UNAM

Manejo de Datos

Facultad de Ciencias

Universidad Nacional Autónoma de México

Tarea: Buenas Prácticas, POO y Principios SOLID en Python

Profesora: Jessica Santizo Galicia

Ayudante: Brenda Lizeth Ramírez Arista

Fecha de salida: 24 de agosto

Fecha de entrega: 8 de septiembre, hasta las 10:00 p.m.

Equipo: Manejadores de Data

Integrantes: Nuñez Martínez Ian Mitl

Tapia Espinoza Diego Josué

Trabajo en Equipo

Objetivo

El alumno pondrá en práctica la resolución de problemas aplicando buenas prácticas de programación y principios SOLID con Python, incluyendo validación robusta de datos, manejo de excepciones, análisis de complejidad algorítmica y comparación empírica de algoritmos.

1. Investigación (2 pts)

1.1 Parte A - PEP8

Revisa la guía de estilo de Python PEP8 (<https://peps.python.org/pep-0008/>) y describe 10 reglas que deberías usar en un programa escrito en este lenguaje. Agrega un ejemplo de código para cada regla (antes/después).

1.2 Parte B - ¿Qué es SOLID?

Investiga y explica con tus propias palabras qué es el conjunto de principios SOLID en el diseño orientado a objetos. Tu investigación debe incluir, para cada uno de los 5 principios:

- Nombre completo del principio y qué problema de diseño busca resolver.
- Una explicación conceptual en tus propias palabras (no copiar textualmente de la fuente).
- Un ejemplo corto en Python que viole el principio (código “malo”).
- La corrección de ese mismo ejemplo aplicando el principio (código “bueno”).
- Al menos una fuente consultada (artículo, video o documentación), citada al final.

Los cinco principios a investigar son:

1. S Single Responsibility Principle (Principio de responsabilidad única)
2. O Open/Closed Principle (Principio de abierto/cerrado)
3. L Liskov Substitution Principle (Principio de sustitución de Liskov)
4. I Interface Segregation Principle (Principio de segregación de interfaces)
5. D Dependency Inversion Principle (Principio de inversión de dependencias)

Investigación

Parte A – PEP 8

1. Sangría o La Regla de los 4 espacios

Explicación:

La sangría es el espacio que se coloca al inicio de una línea de código para indicar que esa línea pertenece a un bloque determinado.

En Python la sangría es importante, puesto que, no es un recurso únicamente estético, sino que, se utiliza para determinar donde empieza y termina un bloque de código.

La recomendación de las PEP 8 es utilizar 4 espacios por cada nivel de sangría. Además, cuando una instrucción es demasiado larga y necesita ocupar varias líneas, las PEP 8 también recomienda utilizar paréntesis, corchetes o llaves, debido a que, Python permite continuar de una manera automática la instrucción.

Para qué es importante:

Consideramos importante la sangría, ya que, permite que el programa sea ordenado y fácil de seguir, pero también hace que Python sepa que instrucciones pertenecen a cada condición, ciclo o función. Por ello, siempre es importante utilizar estos 4 espacios para evitar que diferentes partes del programa tengan una estructura visual confusa.

Código Antes:

```
if edad >= 18
print("Eres mayor de edad")
```

Código Después:

```
if edad >= 18
    print("Eres mayor de edad")
```

2. Longitud Máxima de Línea

Explicación:

Las PEP 8 recomiendan que las líneas de código tengan como máximo 79 caracteres.

Para comentarios y cadenas de documentación (docstrings), recomienda un máximo de 72 caracteres.

La razón principal es para evitar que las líneas lleguen a ser tan largas que el programador tenga de desplazarse horizontalmente o que el editor las divida automáticamente.

Para qué es importante:

Consideramos importante esta regla, ya que, una línea demasiado larga puede hacer que el código sea difícil de leer y revisar. Al dividirla correctamente nos permite observar cada parte de la instrucción sin perdernos.

Código Antes:

```
resultado =  
calcular_promedio_de_calificaciones_de_los_estudiantes(grupo_a, grupo_b, grupo_c)
```

Código Después:

```
resultado =  
calcular_promedio_de_calificaciones_de_los_estudiantes(  
    grupo_a,  
    grupo_b,  
    grupo_c,  
)
```

3. Salto de Línea Antes o Después de un Operador Binario / Regla tradicional Knuth

Explicación:

Los operadores binarios son aquellos que trabajan con dos valores, por ejemplo:

$+$, $-$, $*$, $/$, $=$, $<$, $>$

Cuando una operación matemática o lógica es demasiado larga para una sola línea, existen dos formas de dividirla:

Después del operador:

```
resultado = a +  
            b -  
            c
```

o antes del operador:

```
resultado = (  
    a  
    + b  
    - c  
)
```

Las PEP 8 nos recomiendan para código nuevo el segundo estilo, conocido como regla tradicional de Donald Knuth o simplemente estilo Knuth, porque mantiene el operador junto con los valores que está relacionando.

Para qué es importante:

Consideramos importante la regla tradicional de Knuth, ya que, el objetivo de esta es que, al leer una operación podamos identificar rápidamente que valores se están sumando, restando, multiplicando, etc.

El operador debe quedar visualmente relacionado con los elementos que utiliza, lo que suele dar como resultado un código más legible.

Código Antes:

```
ingreso = (salario + intereses +  
           dividendos - impuestos -  
           gastos)
```

Código Después:

```
Ingreso = (  
    salario  
    + intereses  
    + dividendos  
    - impuestos  
    - gastos  
)
```

4. Importaciones

Explicación:

Las importaciones permiten utilizar módulos, funciones o clases que se encuentran en otras partes de Python o en otros paquetes.

Las PEP 8 recomiendan colocar las importaciones:

- I. Al principio del archivo
- II. Después de los comentarios y docstrings del módulo
- III. Antes de variables y constantes globales

Además, deben organizarse en grupos

A. Biblioteca estándar

```
import os  
import sys
```

B. Bibliotecas externas (relacionadas a terceros)

```
import numpy  
import pandas
```

C. Código propio (específicas de la aplicación/biblioteca local)

```
from mi_proyecto import funciones
```

Para qué es importante:

Consideramos importante esta regla, ya que, una buena organización de las importaciones permite saber de donde viene cada herramienta que estamos utilizando en el programa. También nos evita confusiones entre funciones o clases que tengan nombres iguales.

Código Antes:

```
import sys, os
from math import +
from mi_proyecto import funciones
import numpy
```

Código Después:

```
import os
import sys
from math import sqrt

import numpy

from mi_proyecto import funciones
```

5. Otras Recomendaciones: Espacios alrededor de Operadores Binarios

Explicación:

Las PEP 8 establecen que determinados operadores deben tener un solo espacio a cada lado.

Esto incluye:

Asignación: =

Asignación aumentada: +=, -=, *=

Comparación: ==, !=, <, >, <=, >=

Operadores Lógicos: and, or, not

Operadores de Pertenencia: in, not in

Operadores de Identidad: is, is not

Por ejemplo:
edad = 19

Y no:
edad=19

Para qué es importante:

Consideramos importante esta regla, ya que, los espacios hacen que las operaciones sean más fáciles de identificar visualmente. Es como separar palabras de una oración: el programa puede funcionar sin ellos en algunos casos, pero resulta mucho más sencillo leerlo cuando esta correctamente organizado.

Código Antes:

```
edad=19
nombre="Olaf"
if edad>=18 and nombre= "Olaf":
    print("Puede continuar")
```

Código Después:

```
edad = 19
nombre = "Olaf"
if edad >= 18 and nombre == "Olaf":
    print("Puede continuar")
```

6. Otras Recomendaciones: Estructura de Sentencias Simples y Compuestas

Explicación:

Las PEP 8 recomiendan que cada instrucción tenga su propia línea. Aunque Python permite utilizar “;” para colocar varias instrucciones en una misma línea, al hacerlo puede dificultar la lectura.

Por ejemplo

$x = 10$; $y = 20$; $z = 30$

Es válido, pero no recomendable, en este caso son elementos muy simples, pero al implementarlo en estructuras más complejas con sentencias de varias cláusulas, es cuando aparece el problema.

Debido a lo anterior, se recomienda evitar colocar estructuras complejas completas en una sola línea, especialmente: if/else, for, while; además, se recomienda evitar doblar líneas tan largas.

Para qué es importante:

Consideramos importante esta regla, ya que, al separar las instrucciones hace que el código sea más fácil de leer, modificar y detectar errores. Cuando todo está escrito en una sola línea, es mucho más difícil saber que esta ocurriendo

Código Antes:

```
if edad >= 18: print("Mayor"); else: print("Menor")
```

Código Después:

```
if edad >= 18:  
    print("Mayor");  
else:  
    print("Menor")
```

7. Bloquear Comentarios

Explicación:

Los comentarios de bloque no sirven para explicar una sección completa del código.

Deben colocarse al mismo nivel de sangría que el código que están explicando.

Cada línea usualmente comienza con “#” seguido de un espacio. Si el comentario tiene varios párrafos se puede separar cada párrafo utilizando una línea que contenga siempre un “#”.

Para qué es importante:

Consideramos importante esta regla, ya que, los comentarios ayudan a que otra persona o nosotros mismo al cabo de un rato, podamos entender que pretende hacer una sección del programa.

No se deben explicar cosas tan obvias, sino aportar información útil sobre la intención del código. Un buen comentario explica por qué o para qué se hace algo, no simplemente repetir lo que ya dice el código.

Código Antes:

```
total = matemáticas + física + programación  
promedio = total / 3
```

Código Después:

```
#Calculamos el promedio de las calificaciones.  
#Primero sumamos todas las calificaciones  
#y posteriormente dividimos entre la cantidad de materias  
total = matemáticas + física + programación  
promedio = total / 3
```

8. Diseñando para la Herencia: Atributos Públicos o Privados

Explicación:

Esta regla se relaciona con la Programación Orientada a objetos (POO) y específicamente con la herencia. Cuando creamos una clase, debemos decidir que atributos y métodos estarán disponibles para otras partes del programa y cuales están destinados únicamente al funcionamiento interno de la clase.

Existen principalmente 3 conceptos para los Atributos

- Públicos: son elementos que esperamos que otras partes del programa puedan utilizar

class Persona:

```
    def saludar(self):
        print("Hola")
#saludar( ) es un método público
```

- No públicos (o mal llamados Privados): son elementos que no están pensados para ser utilizados directamente desde fuera de la clase.

class Persona:

```
    def __init__(self):
        self._nombre = "Olaf"
#El "__" indica: Este atributo forma parte de la implementación interna; su uso debe ser cuidadoso
```

- Atributos para subclases (los que forman parte de la API de la subclase): cuando diseñamos una clase pensando en que otras clases heredarán de ella, debemos determinar que elementos pueden utilizar esas subclases.

class Animal:

```
    def hacer_sonido(self):
        print("Sonido")
```

class Perro:

```
    def ladrar(self):
        self.hacer_sonido( )
#hacer_sonido( ) forma parte del comportamiento que puede aprovechar una subclase
```

Para qué es importante:

Consideramos importante esta regla, debido a que, antes de crear una clase, debemos pensar que queremos que otras clases puedan utilizar y que queremos mantener como parte de la implementación interna. Esto nos evita que otras partes del programa dependa de elementos que podríamos cambiar posteriormente.

Código Antes:

```
class Cuenta:

    def _init_(self):
        self.saldo = 1000
        self.calcular_interes = 0
```

Código Después:

```
class Cuenta:

    def _init_(self):
        self.saldo = 1000
        self._interes = 0.05

    def consultar_saldo(self)
        return self.saldo
```

Aquí:

- “ saldo ” público.
- “ _interes ” elemento interno/no público.
- “ consultar_saldo() ” interfaz pública para consultar información.

9. Diseñando para la Herencia:

Explicación:

Cuando una clase esta diseñada para ser heredera y contiene un atributo que no queremos que las subclases utilicen accidentalmente, podemos comenzar su nombre son dos guiones bajos (__). Esto activa un mecanismo en Python llamado modificación de nombres (mangling algorithm).

Por ejemplo:

```
class Padre:
```

```
    def __init__(self):  
        self._dato = 100
```

Internamente, Python modifica el nombre a algo como “_Padre__dato”, esto evita que una subclase utilice accidentalmente un atributo con el mismo nombre.

Ejemplo:

```
class Padre:
```

```
    def __init__(self):  
        self._dato = 100
```

```
class Hijo(Padre):
```

```
    def __init__(self):  
        super().__init__()  
        self._dato = 200
```

Aunque ambos se escriben `_dato`, Python los trata como atributos diferentes:

Padre: `_Padre__dato`

Hijo: `_Hijo__dato`

Por eso reduce las posibilidades de que una subclase sobrescriba accidentalmente un atributo interno de la clase padre.

Para qué es importante:

Consideramos importante esta regla, debido a que, sirve para evitar conflictos accidentales de nombres cuando trabajamos con herencia. No significa que el atributo sea completamente inaccesible, Python realmente no tiene atributos privados absolutos. Es principalmente un mecanismo para evitar interferencias involuntarias.

Código Antes:

```
class Persona:

    def __init__(self):
        self.dato = 100

class Estudiante

    def __init__(self):
        super().__init__()
        self.dato = 200
```

Código Después:

```
class Persona:

    def __init__(self):
        self._dato = 100

class Estudiante

    def __init__(self):
        super().__init__()
        self._dato = 200
```

10. Recomendaciones de Programación:

Explicación:

Python puede ejecutarse mediante diferentes implementaciones, como: CPython, PyPy, Jython, IronPython, Cython, entre otras.

La regla nos indica que debemos escribir código que no dependa de comportamientos internos o características específicas de una implementación concreta.

Un ejemplo clásico es la concatenación repetida de cadenas:

```
a += b
```

En ciertas situaciones, CPython puede optimizar este proceso. Sin embargo, esa optimización no necesariamente existe en otras implementaciones. Para concatenar muchas cadenas, las PEP 8 recomiendan utilizar “ “.join () ”. Esto proporciona un comportamiento de rendimiento más consistente.

Para qué es importante:

Consideramos importante esta regla, debido a que, la idea es que nuestro programa no dependa de un “truco” que solamente funciona bien en una versión o implementación específica de Python.

Así, el código tiene mayores posibilidades de funcionar correctamente si se ejecuta en diferentes entornos.

Código Antes:

```
resultado = ""

for palabra in palabras:
    resultado += palabra
#Aquí estamos concatenando repetidamente una cadena
```

Código Después:

```
resultado = "".join(palabras)

#Si tenemos

palabras = ["Hola", " ", "mundo"]

#obtenemos:
#Hola Mundo

#join( ) esta diseñado específicamente para unir elementos de una secuencia de
#cadenas, por lo que es la opción adecuada cuando necesitamos realizar muchas
#concatenaciones.
```

Parte B – ¿Qué es SOLID?

SOLID es un conjunto de cinco principios de diseño que se utilizan principalmente en la programación orientada a objetos (POO). Tienen como objetivo general ayudar a construir programas que sean fáciles de entender, modificar, mantener y ampliar conforme crecen. El nombre **SOLID**, viene del acrónimo que reúne los principios **S**ingle Responsibility, **O**pen/Closer, **L**iskov Substitution, **I**nterface Segregation y **D**ependency Inversion.

S Single Responsibility Principle (Principio de responsabilidad única)

Problema:

El SRP busca evitar que una sola clase tenga demasiadas responsabilidades diferentes.

El problema aparece cuando una clase se encarga, por ejemplo, de calcular información, guardar datos y generar reportes al mismo tiempo. Si posteriormente cambia cualquiera de esas funciones tendremos que modificar la misma clase.

Esto genera acoplamiento entre tareas que deberían estar separadas y hace que el mantenimiento sea más complicado. La idea fundamental es que una clase tenga u sola razón importante para cambiar.

Explicación Conceptual Propia:

Bajo nuestro criterio, el Principio de Responsabilidad Única (SRP), es una recomendación para implementar si es que el código lo requiere, ya que, su función será dividir y fragmentar las tareas que se llevarán a cabo en el código por las clases; es decir, cada clase deberá concentrarse en una responsabilidad/función/tarea bien definida que le será asignada, mientras que otras clases desarrollarán otras funciones y podrán encargarse de lo demás. Como consecuencia tendremos un orden que permitirá modificar una parte específica del código sin alterar otras funciones/clases. Más que nada es el orden y organización de una clase – una responsabilidad principal – una razón clara para cambiar.

Código Malo:

```
class Empleado:
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario

    def calcular_salario(self):
        return self.salario

    def guardar_en_archivo(self):
        print("Guardando empleado...")

    def generar_reporte(self):
        print("Generando reporte...")

#El problema aquí es que la clase Empleado tiene tres tareas/responsabilidades
#1. Representar al empleado
#2. Guardar información
#3. Generar reportes
```

Código Bueno:

```
class Empleado:
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario

    def calcular_salario(self):
        return self.salario

class GuardarEmpleado:
    def guardar(self, empleado):
        print("Guardando empleado...")

class ReporteEmpleado:
    def generar(self, empleado):
        print("Generando reporte...")

#Ahora cada clase tiene una función específica:
#Empleado = información del empleado
#GuardarEmpleado = guardar información
#ReporteEmpleado = generar reportes
```

O Open/Closed Principle (Principio de abierto/cerrado)

Problema:

El OCP busca evitar que cada vez que agreguemos una nueva funcionalidad tengamos que modificar constantemente código que ya funciona.

El principio establece que una entidad de software debe estar abierta para extensión, pero cerrada a modificación. Es decir. Debemos poder agregar nuevos comportamientos sin tener que alterar continuamente la lógica existente.

Explicación Conceptual Propia:

Bajo nuestro criterio, el Principio de Abierto/Cerrado (OCP) es una recomendación para implementar bastante necesaria en ciertos casos, ya que, está muy relacionada con el principio anterior, básicamente nos dice que una clase debe estar cerrada para su modificación, pero abierta para su extensión.

En otras palabras, si nuestra intención es agregar nuevas funcionalidades al código, no deberíamos cambiar el código que ya validamos, debido a que podríamos generar errores, más bien deberíamos extenderlo desde afuera.

En resumen, el OCP, consiste en que tendremos que diseñar el programa para que podamos agregar nuevas clases o comportamientos sin modificar el código que ya funciona.

Código Malo:

```
class CalculadoraDescuento:
    def calcular(self, tipo_cliente, precio):
        if tipo_cliente == "normal":
            return precio * 0.90

        elif tipo_cliente == "estudiante":
            return precio * 0.80

        elif tipo_cliente == "vip":
            return precio * 0.70

#El problema aparece aquí cuando queremos agregar
#"Cliente nuevo 40%"
#Para esto tendríamos que entrar a modificar la clase
```

Código Bueno:

```
class Descuento:
    def calcular(self, precio):
        return precio

class DescuentoNormal(Descuento)
    def calcular(self, precio):
        return precio * 0.50

class DescuentoEstudiante(Descuento)
    def calcular(self, precio):
        return precio * 0.70

class DescuentoVIP(Descuento)
    def calcular(self, precio):
        return precio * 0.80

#Ahora podemos utilizar cualquiera:

descuento = DescuentoVIP()

precio_final = descuento.calcular(100)

print(precio_final)

#Y si necesitamos un nuevo descuento:

class DescuentoEmpleado(Descuento)
    def calcular(self, precio):
        return precio * 0.60

#No necesitamos modificar las clases anteriores
#Agregamos nuevas funcionalidades mediante extensiones, no modificando
constantemente lo que ya funciona
```

L Liskov Substitution Principle (Principio de sustitución de Liskov)

Problema:

El LSP evita que una subclase herede de otra clase cuando no puede comportarse correctamente como esta clase base.

La idea es que, si una clase B es una subclase de A, un objeto B debería poder utilizarse donde se espera un objeto A sin provocar comportamientos incorrectos.

Explicación Conceptual Propia:

Nota: La creación de un objeto a partir de una clase se le llama instanciación, cada objeto es una instancia de la clase.

Bajo nuestro criterio, el Principio de Sustitución Liskov (LSP) es una recomendación para implementar constantemente, debido a que, si tenemos Clases Padre y Clases Hija, es decir, Herencia, las instancias de ambas deben poder intercambiarse sin que suceda algún resultado inesperado.

Pongamos el ejemplo de la Clase Padre “Ave” y suponemos que todas las aves pueden volar, después creamos Clase Hija “Pingüino”, pero un pingüino no puede volar. Entonces, nuestra Clase Ave esta definiendo algo que no todas sus subclases pueden cumplir.

El problema aquí no es que hayamos programado mal el pingüino. El problema es que la relación de Herencia, la diseñamos incorrectamente.

En resumen, una subclase (Clase Hija) debe ser capaz de sustituir a su clase base (Clase Padre) sin alterar ni comprometer el comportamiento que tenía el programa.

Código Malo:

```
class Ave:
    def volar(self):
        print("Está volando")

class Aguila(Ave):
    pass

class Pinguino(Ave):
    def volar(self):
        raise Exception("Los pingüinos no vuelan")

#Ahora tenemos:

aves = [Aguila( ), Pinguino( )]

for ave in aves:
    ave.volar( )

#Con esto cuando el programa llegue al pingüino, soltará un error
#La subclase Pinguino no puede utilizarse correctamente como cualquier Ave que
#tenga la capacidad de volar
```

Código Bueno:

```
#Podemos sperar el concepto de “ser un ave” del concepto “poder volar”:

class Ave:
    def moverse(self):
        print("El ave se está moviendo")

class AveVoladora(Ave):
    def volar(self):
        print("Está volando")

class Aguila(AveVoladora):
    pass

class Pinguino(Ave):
    pass

#Ahora:

aguila = Aguila()
águila.volar()

pinguino = Pinguino()
pinguino.moverse()

#No estamos obligando al pingüino a implementar algo que no puede hacer
#Llegamos a esta solución, porque la herencia representa correctamente las
#capacidades de cada objeto
#El programa ya no supone que todas las aves vuelan
```

I Interface Segregation Principle (Principio de segregación de interfaces)

Problema:

El ISP busca evitar que una clase sea obligada a implementar métodos que realmente no necesita. La idea es prevenir varias interfaces pequeñas y específicas en lugar de una interfaz enorme que contenga funciones que solamente algunas clases utilizarán.

Python no tiene interfaces exactamente como Java o C, pero podemos representar este concepto mediante clases abstractas utilizando “ABC” y “abstractmethod”.

Explicación Conceptual Propia:

Nota: Las interfaces están muy relacionadas con el polimorfismo (varias formas de hacer lo mismo), pero este polimorfismo tiene el riesgo de traer código inconsistente, ya que, al tener varias formas de hacer lo mismo cada clase vendría con una estructura de código, nombre y argumentos diferentes a las otras, complicando cada vez más el código, para evitar lo anterior se utilizan las interfaces que son como contratos que definen el nombre de los métodos y los argumentos que reciben pero no especifican como implementar el método ese es problema de cada clase.

Además, también son importantes los Clientes que son partes del programa que interactúan/utilizan clases o interfaces.

Bajo nuestro criterio, el Principio de Segregación de Interfaces (ISP) es una recomendación que nos dice los Clientes no deben verse obligados a depender de interfaces que no utilizan, en otras palabras, un Cliente solo debe conocer los métodos que va a utilizar; así, evitamos métodos vacíos, errores innecesarios o funciones que simplemente lanzan excepciones.

En resumen, es mejor tener interfaces pequeñas y específicas que obligar a una clase a implementar funciones que no utiliza.

Código Malo:

```
from abc import ABC, abstractmethod

class Dispositivo(ABC)

    @abstractmethod
    def imprimir(self):
        pass

    @abstractmethod
    def escanear(self):
        pass

    @abstractmethod
    def enviar_fax(self):
        pass

class Impresora(Dispositivo):

    def imprimir(self):
        print("Imprimiendo")

    def escanear(self):
        raise NotImplementedError

    def enviar_fax(self):
        raise NotImplementedError

#La Impresora está obligada a implementar funciones que no necesita
```


Código Bueno:

```
from abc import ABC, abstractmethod

class Imprimible(ABC):

    @abstractmethod
    def imprimir(self):
        pass

class Escaneable(ABC):

    @abstractmethod
    def escanear(self):
        pass

class Impresora(Imprimible):

    def imprimir(self):
        print("Imprimiendo")

class Multifuncional(Imprimible, Escaneable):

    def imprimir(self):
        print("Imprimiendo")

    def escanear(self):
        print("Escaneando")

#Impresora – solamente necesita imprimir
#Multifuncional – puede imprimir y escanear
#Cada clase implementa solamente las capacidades que en realidad necesita
#De esta forma nos evitamos métodos vacíos, errores redundantes o cualquier
#otro inconveniente
```

D Dependency Inversion Principle (Principio de inversión de dependencias)

Problema:

El DIP busca reducir el acoplamiento directo entre clases de alto nivel y clases concretas de bajo nivel. La idea central es que los módulos importantes del programa no deberían depender directamente de implementaciones específicas. Ambos deberían depender de una abstracción.

Explicación Conceptual Propia:

Bajo nuestro criterio, el Principio de Inversión de Dependencias (DIP), es una recomendación que nos dice que las Clases de Alto Nivel no deben depender de las de Bajo Nivel, sino que ambas deben depender de abstracciones y que los detalles a su vez deben depender igualmente de abstracciones, no al revés; dicho de otra forma el objetivo es evitar que una clase quede amarrada a una única forma de hacer algo, en lugar de estar conectadas de manera rígida, las clases deberían relacionarse mediante una abstracción, que funciona como una especie de punto de conexión entre ellas (que tengan menos dependencia entre sí), haciendo que el programa sea más flexible y sencillo de modificar cuando sea necesario.

En resumen, las clases importantes deben depender de abstracciones y no quedar amarradas a una implementación específica.

Código Malo:

```
class Correo:
    def enviar(self, mensaje):
        print("Enviando correo:", mensaje)

class Tienda:
    def __init__(self):
        self.correo = Correo()
    def notificar(self, mensaje):
        self.correo.enviar(mensaje)

#Aquí Tienda está directamente conectada con Correo
#Si quisiéramos cambiar a SMS, debemos modifica Tienda
```

Código Bueno:

```
#Creamos una abstracción

from abc import ABC, abstractmethod

class Notificador(ABC):

    @abstractmethod
    def enviar(self, mensaje):
        pass

#Después creamos implementaciones

class Correo(Notificador):

    def enviar(self, mensaje):
        print("Enviando correo:", mensaje)

class SMS(Notificador):

    def enviar(self, mensaje):
        print("Enviando SMS:", mensaje)

#Y hacemos que Tienda dependa de la abstracción

class Tienda:

    def __init__(self, notificador):
        self.notificador = notificador

    def notificar(self, mensaje):
        self.notificador.enviar(mensaje)
```

```
#Ahora podemos elegir qué sistema utilizar:
```

```
tienda = Tienda(Correo())  
tienda.notificar("Tu pedido está listo")
```

```
"O:
```

```
tienda = Tienda(SMS())  
tienda.notificar("Tu pedido está listo")
```

```
"La Tienda no tuvo que modificarse"
```

Bibliografía

- Alpaca Tech. (2024, enero 16) Principios SOLID explicados con MINECRAFT. YouTube. <https://www.youtube.com/watch?v=dqxeo5WIYP>
- BettaTech. (2020, febrero 10) Los principios SOLID, ¡explicados!. YouTube. <https://www.youtube.com/watch?v=2X50sKeBAcQ>
- Coghlan, A. van Rossum, G., Warsaw, B. (2013, agosto 01) PEP 8 – Guía de estilo para código Python. PEPs Python. <https://peps.python.org/pep-0008/>
- EDteam. (2023, septiembre 29) ¿Qué son los principios SOLID en programación? La mejor explicación en español. YouTube. <https://www.youtube.com/watch?v=g1shhx5Nvv8>
- Equipo Editorial IONOS. (2025, enero 14) SOLID Principles: las cinco reglas de oro de la OOP. IONOS. <https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/solid-principles/>
- Daniel Españadero (2023, abril 16)   ¡PRINCIPIOS SOLID EXPLICADOS! | DE 0 A SUPER SAIYAN 4. YouTube. <https://www.youtube.com/watch?v=KtxNclOV4jk>
- freecodecamp.org, Sabalet, R. D. (2022, noviembre 28) Los principios SOLID de programación orientada a objetos explicados en Español sencillo. freeCodeCamp. <https://www.freecodecamp.org/espanol/news/los-principios-solid-explicados-en-espanol/>
- LenynDev. (2025, enero 20) Todos los principios SOLID con memes y guerreros. YouTube. <https://www.youtube.com/watch?v=gRmLY0GK0Kk>
- Martín, M. J. (2025) SOLID: los 5 principios que te ayudarán a desarrollar software de calidad. Profile. <https://profile.es/blog/principios-solid-desarrollo-software-calidad/>
- Programación y más. (2025, septiembre 23) Aprende SOLID de 1 vez por todas  En 7 min. YouTube. https://www.youtube.com/watch?v=Iu_OH052hBg

Nota: Se hizo uso responsable de AI, como Chat GPT, Gemini, Blackbox, para reforzar conceptos no comprendidos, resolver dudas acerca de los ejemplos de código, verificar la veracidad de argumentos propios del equipo, desarrollar borradores de ejercicios etc.; muchas de las definiciones y conceptos finales implementados vienen básicamente de los vídeos.